09.10.26 Aufgabe 1

1. **Make a Hello World app.** You've already done this, but if you deleted that project, start by creating a new one: **Empty Views Activity**, language **Java**.
2. **Change the text of the "hello_world" TextView from within `onCreate`.** Give the TextView the ID `hello_world` in `activity_main.xml`, then in `MainActivity.java`, after `setContentView`, find it with `findViewById` and change its text with `setText`.
3. **Create an Activity that overrides all state-change methods and logs a message for each one.** Override `onCreate`, `onStart`, `onResume`, `onPause`, `onStop`, `onRestart` and `onDestroy`, call `super` in each, write a `Log.d` message, and watch the messages in Logcat while you press Home, come back, rotate the screen, and so on.

Point 1: empty Hello world

What this guide covers:

- You have downloaded Android Studio and opened it once with the default settings. Nothing else.
- At the end you have an app that shows "Hello World" on a virtual phone.
- Labels you click are in **bold**. Keyboard shortcuts are for the Mac (`Cmd` = Command key).
- The full tour of every button comes in a separate guide. Here you only meet what you need for this app.

1. Before you start

- **Use the uni Wi-Fi for the first run.** The first project and the virtual phone download a lot of data: roughly 1–3 GB in total (Android SDK, Gradle, libraries, phone system image). After that, rebuilding Hello World uses almost no data.
- **Know your Mac's chip.** Apple menu → **About This Mac**. If it says *Apple M1/M2/M3/M4*, the virtual phone must use an `arm64-v8a` image (see Section 8.2). This was probably why your emulator would not start last time.
- **Free disk space:** keep at least 10 GB free.

2. The first launch: Setup Wizard

If you opened Android Studio but did not finish the wizard, it appears again. If you already see the **Welcome to Android Studio** window, skip to Section 3.

1. **Import Android Studio Settings**: choose **Do not import settings**.
2. **Install Type**: choose **Standard**.
3. **License Agreement**: select each item in the list on the left and click **Accept** for each one. **Finish** stays grey until all are accepted.
4. Click **Finish**. Android Studio downloads the *Android SDK* (the tools and libraries to build apps) and the *Emulator*. Wait until it says it is done, then click **Finish** again.

3. Create the project

1. On the **Welcome to Android Studio** window click **New Project**.
   (If a project is already open: menu **File** → **New** → **New Project…**)
2. On the left, keep **Phone and Tablet** selected.
3. Choose the template **Empty Views Activity**, then click **Next**.

> **Careful:** do *not* choose **Empty Activity**. It looks almost the same, but it only works with Kotlin and Jetpack Compose. Your course uses Java, so you need **Empty *Views* Activity**.

Fill in the next screen like this:

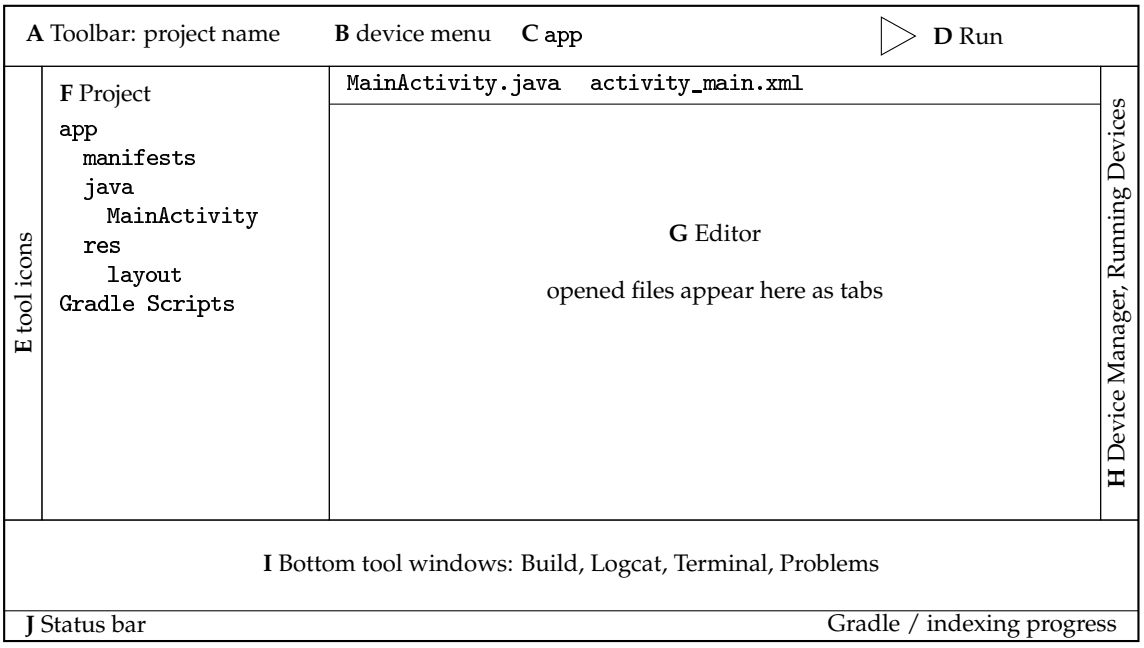
Field	Value	What it means
Name	`HelloWorld`	The app name shown under the icon on the phone.
Package name	`com.example.helloworld`	Unique ID of the app. Filled in automatically; leave it.
Save location	default (`AndroidStudioProjects`)	Folder on your Mac. The default is fine.
Language	**Java**	**Change this.** The default is Kotlin.
Minimum SDK	API 24 (default)	The oldest Android version the app runs on. Leave it.
Build configuration language	Kotlin DSL (default)	Language of the build files only. Leave it; your code is still Java.

Click **Finish**.

4. Wait for the Gradle sync

- *Gradle* is the build tool. It downloads libraries and prepares the project. The first time this can take 5–15 minutes.
- You see progress at the **bottom right** of the window (a small bar and text such as *Gradle sync* or *Indexing*).
- Do not click around or close the window while it runs. It is finished when the bottom bar is quiet and the files on the left are no longer greyed out.
- If a yellow or red bar appears at the top of the editor saying the project needs a sync, click **Sync Now**.

5. Find your way around the window



- **A Toolbar**: at the very top of the window.
- **B Device menu**: chooses which phone (virtual or real) the app runs on.
- **C Run configuration**: says `app`. Leave it.

- **D Run**: the green triangle. Builds the app and starts it on the selected phone. Shortcut `Ctrl`+`R`.
- **E/H Tool icons**: small icons on the left and right edges. Clicking one opens or closes a panel.
- **F Project window**: your files. If it is closed, press `Cmd`+`1`. At the top of this panel there is a dropdown: make sure it says **Android**. This view hides the files you do not need.
- **G Editor**: where you read and write code. Double-click a file in F to open it here.
- **I Bottom panels**: **Build** shows errors when building; **Logcat** shows messages from the running app.
- **J Status bar**: shows background work such as the Gradle sync.

On a Mac the menus (**File**, **Edit**, **View**, **Build**, **Run**, **Tools**…) are in the macOS menu bar at the top of the *screen*, not inside the window.

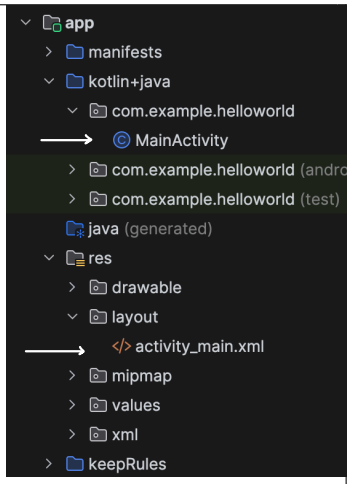
6. The three files of the app

The template has already written a complete Hello World app. You do not have to type anything. Here is what each file does.

6.1 `app/java/com.example.helloworld/MainActivity.java`

An *Activity* is one screen of an app. This class is the code that runs when the screen opens.

```
1  package com.example.helloworld;
2
3  import android.os.Bundle;
4
5  import androidx.activity.EdgeToEdge;
6  import androidx.appcompat.app.AppCompatActivity;
7  import androidx.core.graphics.Insets;
8  import androidx.core.view.ViewCompat;
9  import androidx.core.view.WindowInsetsCompat;
10
11 public class MainActivity extends AppCompatActivity {
12
13     @Override
14     protected void onCreate(Bundle savedInstanceState) {
15         super.onCreate(savedInstanceState);
16         EdgeToEdge.enable(this);
17         setContentView(R.layout.activity_main);
18         ViewCompat.setOnApplyWindowInsetsListener(findViewById(R.id.main), (v, insets) -> {
19             Insets systemBars = insets.getInsets(WindowInsetsCompat.Type.systemBars());
20             v.setPadding(systemBars.left, systemBars.top, systemBars.right, systemBars.bottom);
21             return insets;
22         });
23     }
24 }
```



- **Line 11**: `MainActivity` *is a* screen, because it extends `AppCompatActivity`.
- **Line 14**: `onCreate` is called by Android when the screen is created. You never call it yourself.
- **Line 17**: `setContentView(R.layout.activity_main)` is the important line. It says: "show the layout described in `activity_main.xml`". `R` is a class Android generates automatically with an ID for every resource file.
- **Lines 16 and 18–22**: make the app draw behind the status bar and then add padding so the text is not hidden under it. Not important for now; you can ignore them.
- Your exact template may differ by a line or two between Android Studio versions. That is normal.

Course note. Mobile Systeme grades readability with Sun-based conventions, including a maximum line length of 80 characters. The template's lines 18–20 in `MainActivity.java` are longer than that. It does not matter for Hello World, but in your project you will have to break such lines.

6.2 app/res/layout/activity_main.xml

The *layout*: what is on the screen and where. Open it and click **Code** at the top right of the editor to see the XML (**Design** shows a preview, **Split** shows both).

```
1  <?xml version="1.0" encoding="utf-8"?>
2  <androidx.constraintlayout.widget.ConstraintLayout
3      xmlns:android="http://schemas.android.com/apk/res/android"
4      xmlns:app="http://schemas.android.com/apk/res-auto"
5      xmlns:tools="http://schemas.android.com/tools"
6      android:id="@+id/main"
7      android:layout_width="match_parent"
8      android:layout_height="match_parent"
9      tools:context=".MainActivity">
10
11     <TextView
12         android:layout_width="wrap_content"
13         android:layout_height="wrap_content"
14         android:text="Hello World!"
15         app:layout_constraintBottom_toBottomOf="parent"
16         app:layout_constraintEnd_toEndOf="parent"
17         app:layout_constraintStart_toStartOf="parent"
18         app:layout_constraintTop_toTopOf="parent" />
19
20 </androidx.constraintlayout.widget.ConstraintLayout>
```

- **Line 2**: a `ConstraintLayout` is a container that places things by "attaching" them to edges. `match_parent` = fill the whole screen.
- **Line 6**: `@+id/main` gives the container the name `main`. This is the `R.id.main` used on line 18 of the Java file.
- **Line 11**: a `TextView` is a piece of text on the screen.
- **Line 14**: the text itself. **This is your Hello World.**
- **Lines 15–18**: attach the text to all four edges, which centres it.

6.3 app/manifests/AndroidManifest.xml

The app's ID card for Android: name, icon, and which Activity opens first. The important part is the `<intent-filter>` with `MAIN` and `LAUNCHER` inside the `<activity>` entry of `.MainActivity`: it means "when the user taps the icon, open `MainActivity`". You do not need to change anything here.

7. Optional: make it your own text

1. Open `activity_main.xml` in **Code** view.
2. On line 14 change `"Hello World!"` to, for example, `"Hello World from Martina"`.
3. Saving is automatic in Android Studio. There is no Save button you need to press.

A small yellow warning on that line (*Hardcoded string*) is normal. Android prefers to keep text in a separate file,
`res/values/strings.xml`, which matters for translations. You can ignore it for Hello World.

8. The virtual phone (emulator)

The *emulator* is a phone that runs as a window on your Mac. One virtual phone is called an *AVD* (Android Virtual Device).

8.1 Check if one already exists

- Look at the device menu (**B**) in the toolbar. Recent versions create one automatically with the Standard setup, often called *Medium Phone API 36* or similar.
- Or open **Device Manager** with the phone icon on the right edge (**H**), or **View** → **Tool Windows** → **Device Manager**.
- If a device is listed, go to Section 9.

8.2 If there is none: create one

1. In **Device Manager** click **+** → **Create Virtual Device**.
2. Choose a phone, e.g. **Pixel 8** or **Medium Phone**. Click **Next**.
3. Choose a system image (the Android version). Pick the one marked recommended, with the newest API level.
   - On an Apple M-chip Mac, the ABI column must say `arm64-v8a`. **Not** `x86_64`.
   - Click the download icon next to it if it is not downloaded yet (around 1–2 GB, uni Wi-Fi).
4. Click **Finish**.

9. Run the app

1. Select your virtual phone in the device menu (**B**).
2. Click the green **Run** triangle (**D**), or Ctrl+R.
3. The **Build** panel shows progress at the bottom. The first build takes a few minutes.
4. The phone starts in the **Running Devices** panel on the right side of Android Studio. The first boot can take 1–3 minutes, like a real phone turning on.
5. The app opens by itself: a white screen with **Hello World!** in the middle and *HelloWorld* in the top bar. Done.

Keep the emulator open. When you change the code, just click **Run** again. The app reinstalls in seconds without rebooting the phone. To stop the app: the red square **Stop** button next to **Run**.

10. If something goes wrong

Problem	What to do
Emulator very slow or never starts	• Check the system image is `arm64-v8a` (Section 8.2). • **Device Manager** → three dots next to the device → **Cold Boot Now**. • Still stuck: same menu → **Wipe Data**, then try again. • Close other heavy apps; the emulator needs RAM.
"Gradle sync failed"	• Check you are online, then click **Try Again** or the **Sync Project with Gradle Files** button (elephant icon, top right). • Read the first red line in the **Build** panel; that is the real error.
Code shown in Kotlin (`.kt` files)	• You chose Kotlin or the wrong template. Create a new project: **Empty Views Activity** + **Java**.
Red text everywhere in the code	• Usually the sync is still running. Wait until the status bar is quiet.
Device menu says "No Devices"	• Create a virtual phone (Section 8.2).

Alternative: a real Android phone

Often faster than the emulator, and it uses no download for a system image.

1. On the phone: **Settings** → **About phone** → tap **Build number** 7 times. This unlocks **Developer options**.
2. In **Developer options** turn on **USB debugging**.
3. Connect the phone with a USB cable and accept the "Allow USB debugging?" message on the phone.
4. The phone appears in the device menu (**B**). Select it and click **Run**.

11. Words to know

Term	Meaning
Activity	One screen of the app, written as a Java class (`MainActivity`).
Layout	An XML file in `res/layout` that describes what is on a screen.
View	Anything visible on the screen, e.g. `TextView`, `Button`.
`R`	Auto-generated class with an ID for every resource (`R.layout.activity_main`, `R.id.main`).
Manifest	`AndroidManifest.xml`: app name, icon, permissions, start screen.
Gradle	The build tool. Turns your code into an installable app.
SDK / API level	The Android libraries and their version number (API 36 = a recent Android).
Emulator / AVD	A virtual phone on your Mac.
APK	The finished app file that gets installed on the phone.
Logcat	Live log of messages from the running app; useful for debugging later.

Part 2: change the TextView in onCreate

Step 1: Give the TextView an ID

1. In the **Project** panel on the left, open **app → res → layout → activity_main.xml**.
2. It probably opens in **Design** view (a picture of the phone screen). Click **Code** in the top-right corner of the editor to see the XML.
3. Find the `<TextView` element. It looks roughly like this:

```xml
<TextView
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Hello World!"
    app:layout_constraintBottom_toBottomOf="parent"
    ... />
```

4. Add this line right after `<TextView`:

```xml
android:id="@+id/hello_world"
```

This creates the ID, and Android Studio generates the Java constant `R.id.hello_world` from it, which is how your Java code will find this TextView.

Step 2: Change the text from Java

1. Open **app → java → com.example.... → MainActivity.java**.
2. Find the `onCreate` method and the line `setContentView(R.layout.activity_main);`.
3. Directly **below** that line, add:

```java
TextView helloText = findViewById(R.id.hello_world);
helloText.setText("Text changed from Java!");
```

The first line finds the TextView in the loaded layout and stores it in a variable. The second line replaces its text. It must come after `setContentView`, because before that line the layout isn't loaded yet.

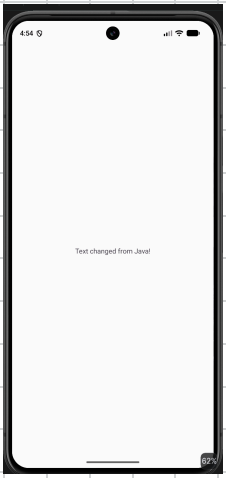
Also `import android.widget.TextView;` in the import section



From epigenetics: phenotype = changes you can see

genotype = changes in the code

Phenotype: the text in the midde of the app has gone from "Hello World" to "Text changed from Java"



Genotype:

- In the layout (`activity_main.xml`), you gave the TextView an **ID**, `hello_world`. From this, Android Studio generated the constant `R.id.hello_world`, which lets the Java code refer to that specific view.
- In the Activity (`MainActivity.java`), inside `onCreate` and after the layout is loaded with `setContentView`, you used `findViewById(R.id.hello_world)` to get a **reference** to the TextView as a Java object. Then you called `setText(...)` on it to change its text at runtime.

In one sentence: you connected a view defined in the XML layout to the Java code through its ID, so the Activity can now change that view while the app is running.

↳ doesn't mean "during run time of the code"

Think of it as two moments in time.

Before the app runs: the XML is a fixed description, like a printed sign. It says "Hello World!", and it can't change itself.

When the app runs: when you open the app, Android executes `onCreate` line by line:

1. `setContentView(...)` loads the layout, so the screen now has the TextView with "Hello World!".
2. `findViewById(...)` gets the TextView.
3. `setText(...)` replaces the text with "Text changed from Java!".

All of this happens in a fraction of a second, so you never actually see "Hello World!".

So "while the app is running" just means the text is changed **by code that executes inside the app after it has started**, instead of being fixed in advance in the XML.

In an actual app:
xml defines the structure of the fixed text
and Java sets the text when it depends on other information

Part 3 : add the 7 lifcycles to your Activity

↑ MainActivity

Step 1: Add a TAG constant

Open `MainActivity.java`. Directly below the line `public class MainActivity extends AppCompatActivity {`, add:

```java
private static final String TAG = "Lifecycle";
```

This is the label every log message will have, so you can filter for it in Logcat later.

Step 2: Log in `onCreate`

`onCreate` already exists. At the end of it, just before its closing `}`, add:

```java
Log.d(TAG, "onCreate");
```

`Log` will turn red. Click on it, press ⌥ **Option** + **Enter** and choose **Import class**, which adds `import android.util.Log;` at the top.

Step 4: Run and open Logcat

1. Press ▶ **Run**.
2. Open **Logcat**: it's the cat icon in the bottom-left toolbar, or **View → Tool Windows → Logcat**.
3. In Logcat's search bar, type `tag:Lifecycle`, so only your messages are shown.

You should already see `onCreate`, `onStart` and `onResume`.

Step 5: Trigger the state changes

Do these in the emulator and watch Logcat after each one:

Action in the emulator	What you should see
Start the app	onCreate → onStart → onResume
Press Home (the circle button)	onPause → onStop
Reopen the app from the app list or recents	onRestart → onStart → onResume
Rotate the screen (rotate buttons in the emulator's side toolbar)	onPause → onStop → onDestroy → onCreate → onStart → onResume
Press Back	onPause → onStop (no onDestroy on Android 12+)
Open recents and swipe the app away	onDestroy, if it hasn't been called already

If rotation does nothing, the emulator's auto-rotate may be off. Swipe down from the top of the emulator screen and turn on **Auto-rotate**.

Step 3: Add the other six methods

Below the closing `}` of `onCreate`, but still inside the class (before the last `}` of the file), paste:

```java
@Override
protected void onStart() {
    super.onStart();
    Log.d(TAG, "onStart");
}

@Override
protected void onResume() {
    super.onResume();
    Log.d(TAG, "onResume");
}

@Override
protected void onPause() {
    super.onPause();
    Log.d(TAG, "onPause");
}

@Override
protected void onStop() {
    super.onStop();
    Log.d(TAG, "onStop");
}

@Override
protected void onRestart() {
    super.onRestart();
    Log.d(TAG, "onRestart");
}

@Override
protected void onDestroy() {
    super.onDestroy();
    Log.d(TAG, "onDestroy");
}
```

Phenotype : nothing's changed

Genotype :

- You **overrode** the seven lifecycle callbacks in `MainActivity` : `onCreate`, `onStart`, `onResume`, `onPause`, `onStop`, `onRestart` and `onDestroy`. Android calls these automatically whenever the Activity changes state.
- Each one first calls its `super` version, so Android can still do its own work, and then writes a debug message with `Log.d`.
- You added a `TAG` constant, `"Lifecycle"`, which labels every message so you can filter them in Logcat, plus the import for `android.util.Log`.